

How Disney+ Hotstar Managed (5cr) + Live Viewers During India's T20 World Cup Win 2024

DAY 119
06/Nov/2025

In the Recent Mens T20 cricket World cup 2024, Disney+ Hotstar Reached a Whooping record viewership of 5cr+.

But in the online streaming industry, where some platforms crash even for 3cr viewers, how did hotstar managed to scale even for such a large count of users?

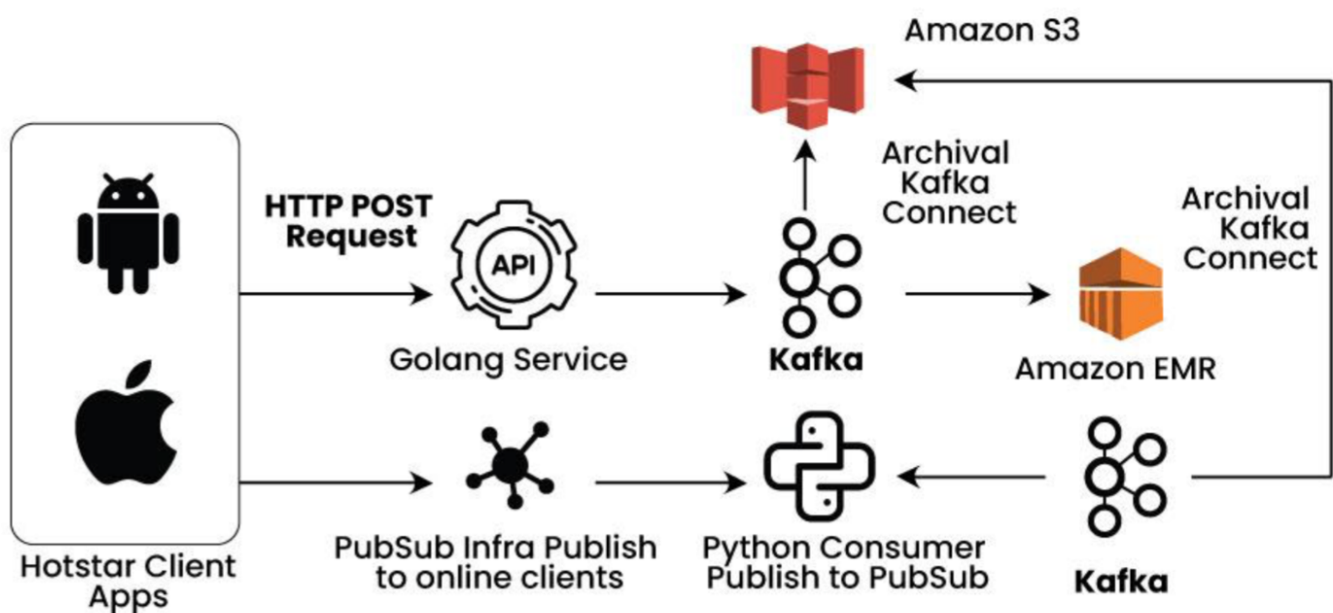
What is Disney+ Hotstar?

Disney+ Hotstar is an **online video streaming platform** owned by Novi Digital Entertainment Private Limited, a wholly-owned subsidiary of Star India Private Ltd. Disney+ Hotstar has a ton of different things to watch like sports, popular TV shows, movies, live streaming, and special shows.

How did Disney+ Hotstar manage 5 Cr+ viewers during the live stream?

Disney+ Hotstar use robust backend infrastructure for handling the live stream, they follows effective backend architecture:

- They have taken the help of third party like **AWS S3**, AWS S3 handles all the data during streaming.
- CDN they use **Akamai**, which optimize the media files for delivering, by ensuring low latency and high performance during streaming. They encode their data in bits so it never fails during live streams.
- They use strong microservice architecture, and for handling the data they use a load balancer. So it can easily handle large viewership at the same time.



Disney+ Hotstar write there code in Go Lang, because Go Lang handle the data easily, and it's easy for developer to understand and write the code. For Go Lang they don't need to pay, and it's easy to resolve bugs for application . Disney+ Hotstar have used effective System Design for handling their live stream. Their are some following ways how they have handle there data:

How using Kafka made a difference for Disney+ Hotstar?

How using Kafka made a difference for Disney+ Hotstar :

- Kafka plays an important role in Disney+ Hotstar during live stream. It helps to process and move data from one system to another, and it consumes streams of data.
- Kafka's distributed architecture allows for [horizontal scalability](#). This is crucial for handling varying levels of demand during live-streaming events, where there may be sudden spikes in viewership.
- Apache Kafka is massively scalable because it allows data to be distributed across multiple servers, and it's extremely fast because it decouples data streams, which results in [low latency](#). It can also distribute and replicate partitions across many servers, which protects against server failure.

How is Hotstar's CDN platform different from others?

- Disney+ Hotstar works on a cloud computing platform to scale its infrastructure. Cloud Services, such as Amazon Web Services, Microsoft Azure, or Google Cloud Platform, allow platforms like Disney+ Hotstar to scale resources up or down as viewership fluctuates.
- They use a robust [CDN](#) to efficiently deliver content to users across the globe.
- CDNs [cache](#) and distribute content to servers strategically located in various geographic locations, reducing latency and ensuring the fastest content delivery.

Hotstar has 500+ AWS CPU instances, which are C4.4X Large or C4.8X Large running at 75% utilization. C4.4X instances have typically 30 Gigs of RAM & C4.8X 60 Gigs of RAM!. The entire setup of Disney+ Hotstar infrastructure has 16 TBs of RAM, 8000 CPU core, with a peak speed of 32Gbps for data transfer. This is the scale of their operations, which ensures that millions of users are able to concurrently access live streaming on their app.

[Microservice architecture](#) and their role in live streams

- [Microservices architecture](#) used by Disney+ Hotstar. Companies, especially those in the streaming industry, often employ microservices architecture to enhance scalability, flexibility, and maintainability.
- Microservices architecture is a design approach where an application is developed as a collection of small, independent services, each focused on a specific business capability. These services communicate with each other through well-defined APIs.
- For microservices architecture enables platforms to break down their applications into smaller, independent services. Each microservice can handle specific functionalities, making it easier to scale individual components based on demand and maintain agility.

[Load Balancer](#) - The Star of Hotstar's infrastructure

Load Balancing helps in distributing incoming network traffic across multiple servers to ensure that no single server becomes overwhelmed. This helps maintain system performance, especially during peak times when there is high volume of concurrent users.

- To distribute incoming traffic across multiple targets, such as EC2 instances, containers, and IP addresses, Disney+ Hotstar utilized Amazon ELB.
- This service ensured that no single server became overwhelmed with too many requests, maintaining a high availability and fault tolerance for their application.

[Redis](#) on Amazon ElastiCache

To handle caching and fast access to data, Disney+ Hotstar could have used Redis on Amazon ElastiCache. This in-memory data store and cache service provides sub-millisecond latency, which is crucial for delivering real-time updates and responses.

Innovative Video Compression Techniques

Disney+ Hotstar employed cutting-edge video compression algorithms to reduce the bandwidth required for high-definition streaming.

- Techniques like adaptive bitrate streaming (ABR) ensured that users with varying internet speeds received the best possible video quality without buffering.
- By optimizing the video delivery process, they were able to serve more users simultaneously without compromising on quality.

Global CDN Partnerships

Disney+ Hotstar partnered with leading global [CDN](#) providers to distribute content efficiently across the globe.

- These partnerships ensured that content was cached and delivered from servers closest to the viewers, reducing latency and improving load times.
- The platform's strategic use of multiple CDN providers also added redundancy, ensuring that even if one network faced issues, others could seamlessly take over.

How Disney+ Hotstar store the data in there Database?

- For database management is crucial for handling large amounts of user data, so Hotstar+Disney use [NOSQL databases](#), caching mechanisms and database sharding these are common strategies to manage and scale database.
- As per network condition users receive the appropriate quality content beacuse content is encoded in multiple quality levels. During live stream the content is encoded in bitrate.

Backend systems are equipped with monitoring tools and analytics to track user behavior, system performance, and potential issues. This data helps the platform optimize its services and user experience.

Why did other streaming platforms crash during livestream but Disney+ Hotstar didn't fail?

Let's see how other streaming platforms crash during live stream:

- Other Streaming Platforms use core technologies such as Javascript, Node.JS, Kafka, and others, alongside cloud infrastructure tools like Kubernetes, Jenkins, Kibana, and others.
- They need to spend enough time and budget to deliver an effective system design.
- Having a cloud provider is not enough—the real skill lies in how the services offered by the cloud are effectively integrated into the business outcomes of the platform.

Disney+ Hotstar leveraged an advanced combination of multi-tiered auto-scaling and edge computing to handle 5 Cr+ concurrent viewers during India's T20 World Cup victory. Here's how these technologies worked together to ensure a flawless streaming experience:

1. Multi-Tiered [Auto-Scaling](#):

- **Dynamic Resource Allocation:**
 - Disney+ Hotstar's infrastructure dynamically adjusted its resources in real-time based on viewer demand.
 - This was not just limited to adding more servers but involved scaling different components of the system like [load balancers](#), databases, and [content delivery networks \(CDNs\)](#) simultaneously.
 - This multi-tiered approach ensured that every layer of the streaming pipeline could handle the surge in traffic without bottlenecks.
- **Predictive Analytics:**
 - Using machine learning algorithms, the platform predicted traffic spikes based on factors such as match progress, user engagement patterns, and social media trends.
 - This predictive capability allowed them to preemptively scale their infrastructure ahead of peak demand, ensuring

seamless service even during the most intense moments of the match.

2. Edge Computing:

- **Localized Processing:**

- By deploying edge servers closer to users, Disney+ Hotstar minimized latency and reduced the load on central servers.
- These edge servers handled tasks like caching popular content, processing user requests, and even managing some aspects of video encoding.
- This distributed approach meant that user requests were processed faster and more efficiently, contributing to a smoother streaming experience.

- **Real-Time Analytics at the Edge:**

- The platform also utilized edge computing to perform real-time analytics and monitoring.
- This allowed for immediate detection and resolution of issues at the local level before they could impact the overall viewing experience.
- By addressing problems such as network congestion or server overloads locally, Disney+ Hotstar maintained a high-quality stream for all users.

Key Changes that Disney+ Hotstar made to ensure Scalability

Disney+ Hotstar for live streaming they follow effective system designs:

- Hotstar built faster streaming pipelines for consumption analytics, else it will directly affect the quality of the broadcast.
- They have ensure that all bad data is discarded—that is, if there are too many bad events, it will impact the analytics, which can further lead to hampering key data-driven business decisions.
- Next, they have proper metrics used for autoscaling. In the event that this is not the case, the application will fail to respond to sudden load.
- They have used microservices and event-driven architectures with message brokers like Kafka.
- They use decouple applications so there is minimum impact of one system to another.